

La ricerca locale PG (Potential Game)

spiegata in modo semplice, con un esempio passo per passo

DFaaSOptimizer

9 ottobre 2026

1 L'idea in tre righe

Ogni nodo edge riceve richieste per alcune funzioni serverless. Può *eseguirle da sé, passarle a un vicino o mandarle al Cloud*. PG è una **ricerca locale a turni**: i nodi, uno alla volta, provano a cambiare la propria scelta; il cambio viene **accettato solo se migliora** il guadagno del nodo. Poiché la somma dei guadagni di tutti i nodi (il *potenziale* Φ) sale a ogni mossa accettata, il procedimento **non può girare in tondo** e prima o poi si ferma in un equilibrio in cui nessuno ha più convenienza a cambiare (equilibrio di Nash approssimato, o ε -Nash).

2 Gli ingredienti

Per un nodo i e una funzione f :

Simbolo	Significato
λ_i	richieste al secondo che arrivano al nodo (carico)
x_i	richieste eseguite localmente
y_{ij}	richieste mandate dal nodo i al vicino j
$z_i = \lambda_i - x_i - \sum_j y_{ij}$	richieste che finiscono nel Cloud
r_i	repliche (container) della funzione attive sul nodo
$\kappa = u_{\max}/d$	richieste che una replica sostiene (qui $0.8/0.1 = 8$)
M_i, m	memoria del nodo e memoria richiesta da una replica
$\rho_i = M_i - m r_i$	memoria libera (<i>slack</i>) del nodo
$\alpha, \beta_{ij}, \gamma$	premio per eseguire in locale, premio per inoltrare a j , penale Cloud

Guadagno di un nodo. Ogni nodo valuta la propria situazione con

$$u_i = \frac{\alpha x_i + \sum_j \beta_{ij} y_{ij} - \gamma z_i}{\lambda_i} \quad (\text{compute_node_utility})$$

cioè: guadagno pieno per ciò che esegue, guadagno un po' minore per ciò che fa eseguire ai vicini, una piccola perdita per ciò che manda al Cloud.

Potenziale. $\Phi = \sum_i u_i$ è esattamente l'obiettivo del problema centralizzato (`compute_centralized_objective`). Quando un nodo cambia solo le *proprie* scelte e non tocca quanto già promesso agli altri, la variazione di Φ coincide con la variazione del suo u_i : migliorare se stessi significa migliorare il sistema.

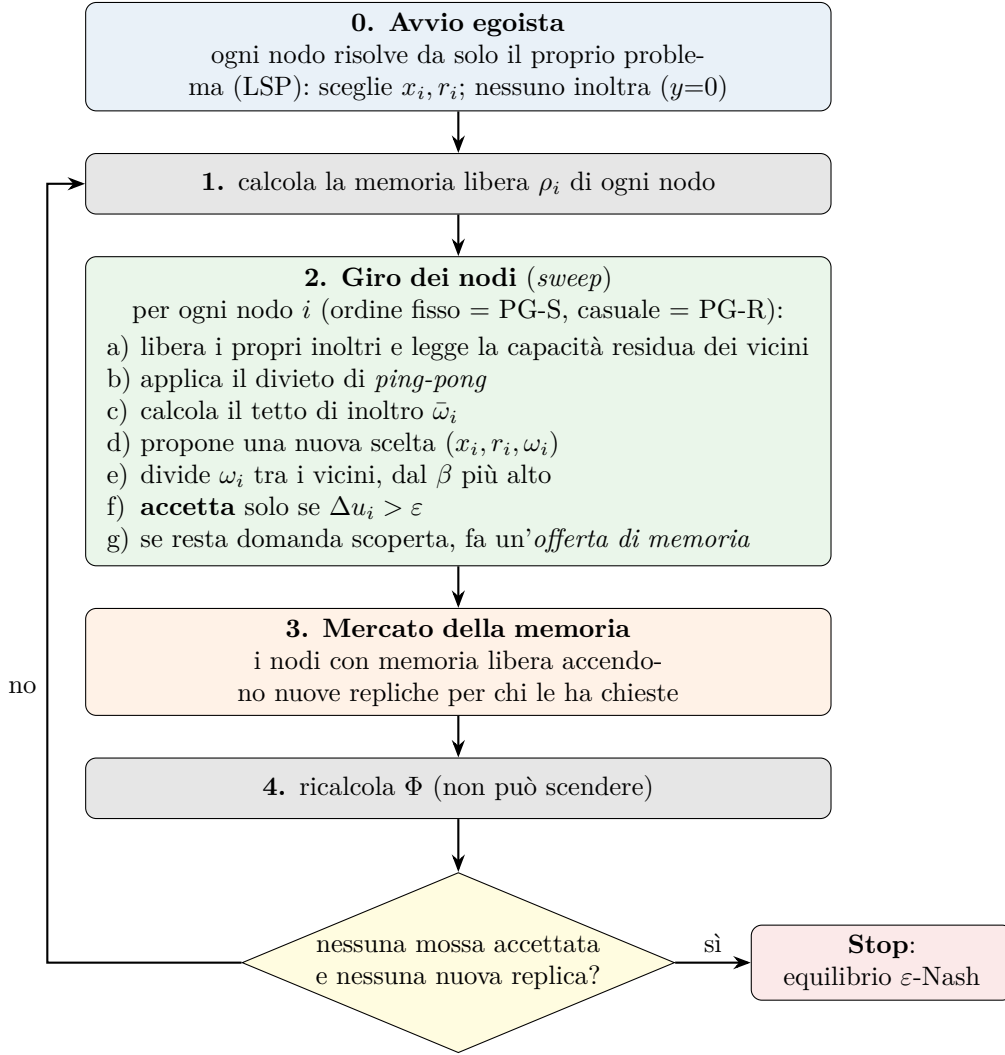


Figura 1: Schema di PG (`decentralized_potentialgame.py`, funzione `_run`). Ci si ferma anche al numero massimo di iterazioni o al limite di tempo.

3 L'algoritmo passo per passo

Cosa succede in ogni passo

- 0. Avvio egoista.** Ogni nodo massimizza il proprio guadagno ignorando gli altri. Esegue in locale quanto può con la memoria che ha; il resto, non potendo ancora contare sui vicini, va al Cloud.
- 1. Memoria libera.** $\rho_i = M_i - m r_i$, ricalcolata a ogni giro perché le mosse accettate cambiano le repliche (`compute_rho`).
- 2a. Libera e guarda.** Il nodo i “ritira” per un attimo i propri inoltri e guarda quanta capacità avanza ai vicini: $\text{residuo}_j = \kappa r_j - (\text{carico già servito da } j)$ (`compute_residual_capacity`). Questo è il *registro* (ledger).
- 2b. Niente ping-pong.** Un nodo non può insieme mandare e ricevere la stessa funzione. Quindi: un vicino che sta già inoltrando f non può riceverne; e chi riceve f da altri non può inoltrarla.
- 2c. Tetto di inoltri.** $\bar{\omega}_i = \text{somma dei residui dei vicini per cui inoltrare conviene più del Cloud } (\beta_{ij} > -\gamma)$.

- 2d. Proposta.** Il nodo risolve il suo piccolo problema locale (LSP_pg, con programmazione dinamica se use_dp è attivo): deve continuare a servire ciò che ha promesso agli altri e non può inoltrare più di $\bar{\omega}_i$. In questa fase l'inoltro è pagato con un prezzo medio δ .
- 2e. Divisione golosa.** La quantità ω_i viene distribuita tra i vicini a partire da quello con il β_{ij} più alto, riempiendone il residuo, poi il successivo, e così via (split_omega). Ciò che non entra va al Cloud.
- 2f. Accettazione.** Si ricalcola il guadagno *vero* (con i β_{ij} effettivi). Se $u_i^{\text{nuovo}} - u_i^{\text{vecchio}} > \varepsilon$ la mossa diventa definitiva, altrimenti si butta via.
- 2g. Offerta di memoria.** Se al nodo resta domanda scoperta (parte di ω_i non piazzata, oppure richieste ancora nel Cloud), chiede ai vicini che hanno memoria libera di accendere repliche per lui.
- 3. Mercato della memoria.** Ogni nodo con memoria libera la divide tra le funzioni richieste in proporzione alle offerte e accende quante più repliche possibile (start_additional_replicas). Così, al giro successivo, i vicini hanno più capacità residua da offrire.
- 4. Controllo.** Φ non deve mai diminuire. Se in un giro intero nessuna mossa è stata accettata e il mercato non ha acceso repliche, nessun nodo può migliorare: siamo in equilibrio e ci si ferma (check_pg_stopping).

4 Esempio numerico e grafico

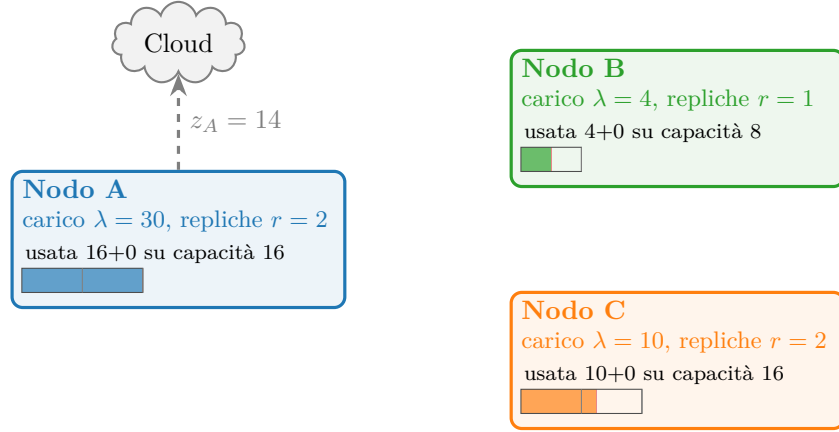
Tre nodi tutti vicini tra loro, una sola funzione. Ogni replica occupa 1 unità di memoria e sostiene $\kappa = 8$ richieste/s. $\alpha = 1$, $\gamma = 0.1$, $\delta = 0.9$, $\varepsilon = 10^{-6}$, ordine fisso A, B, C (PG-S).

	Nodo A	Nodo B	Nodo C
carico λ	30	4	10
memoria M (= repliche massime)	2	4	2
premio β verso il 1° vicino	$\beta_{AB} = 0.9$	$\beta_{BA} = 0.9$	$\beta_{CB} = 0.9$
premio β verso il 2° vicino	$\beta_{AC} = 0.8$	$\beta_{BC} = 0.8$	$\beta_{CA} = 0.8$

A è sovraccarico: con 2 repliche sostiene solo 16 delle sue 30 richieste. B ha poco lavoro e molta memoria; C ha un po' di capacità avanzata. Tutti i numeri che seguono sono stati ottenuti eseguendo le funzioni del codice (node_move, split_omega, start_additional_replicas) su questa istanza.

Come leggere le figure. Nella barra di ogni nodo ogni tacca è una replica (8 richieste/s); il colore del nodo indica il carico proprio eseguito (x), il **rosso** il carico ricevuto dai vicini, il bianco la capacità ancora libera.

Passo 0 — Avvio egoista



$$\Phi = 0.487 + 1 + 1 = 2.487$$

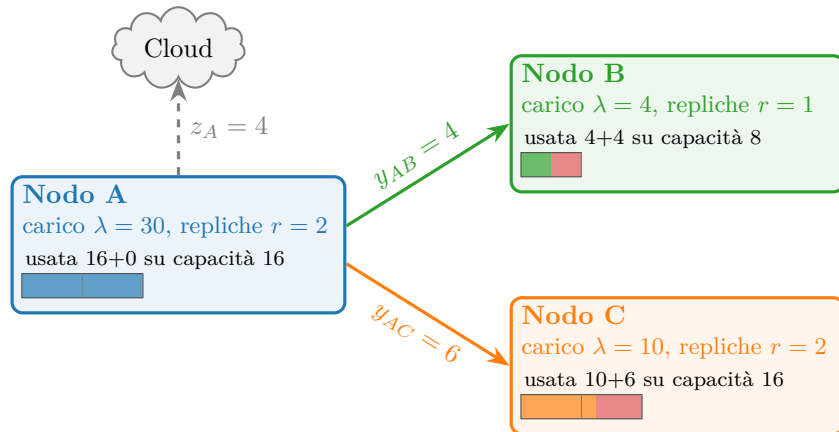
Ogni nodo accende le repliche che gli servono e che la memoria permette: A $r = 2$ (esegue 16, manda 14 al Cloud), B $r = 1$, C $r = 2$.

$$u_A = \frac{16 - 0.1 \cdot 14}{30} = \frac{14.6}{30} = 0.487, \quad u_B = \frac{4}{4} = 1, \quad u_C = \frac{10}{10} = 1.$$

Memoria libera: $\rho = (0, 3, 0)$.

Giro 1, mossa di A — primi inoltri

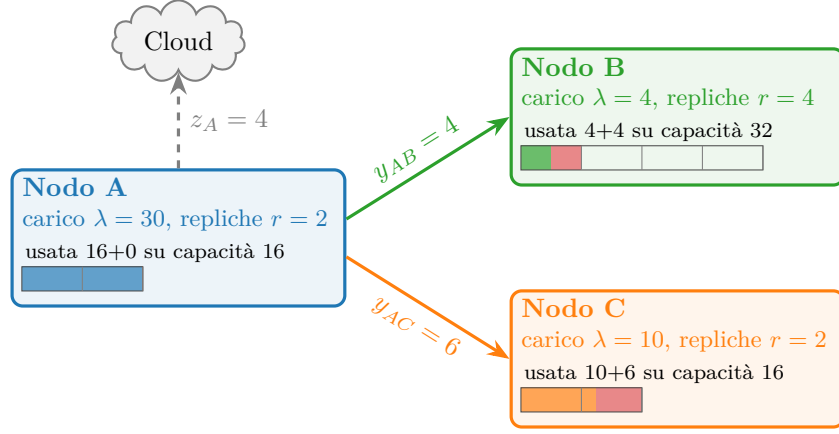
- Residui dei vicini: B = $8 - 4 = 4$, C = $16 - 10 = 6$.
- Nessuno inoltra ancora: niente divieti.
- Tetto: $\bar{\omega}_A = 4 + 6 = 10$.
- Proposta: $x_A = 16$, $r_A = 2$, $\omega_A = 10$ (vorrebbe inoltrarne 14, ma il tetto è 10).
- Divisione golosa: prima B ($\beta = 0.9$) riceve 4 e si riempie, poi C ($\beta = 0.8$) riceve 6. Restano 4 richieste al Cloud.
- Nuovo guadagno $u_A = \frac{16 + 0.9 \cdot 4 + 0.8 \cdot 6 - 0.1 \cdot 4}{30} = \frac{24}{30} = 0.800$; $\Delta u_A = 0.313 > \varepsilon \Rightarrow$ **accettata**.
- A ha ancora 4 richieste nel Cloud: fa un'offerta di memoria a B (che ha $\rho_B = 3 \geq 1$). C non ha memoria libera.



$$\Phi = 0.800 + 1 + 1 = 2.800 \quad (\text{offerta di memoria A} \rightarrow \text{B})$$

Mosse di B e C. Entrambi ora *ricevono* la funzione da A, quindi per il divieto di ping-pong non possono inoltrarla ($\bar{\omega} = 0$). La loro migliore proposta è lasciare tutto com'è: $\Delta u = 0$, **rifutate**.

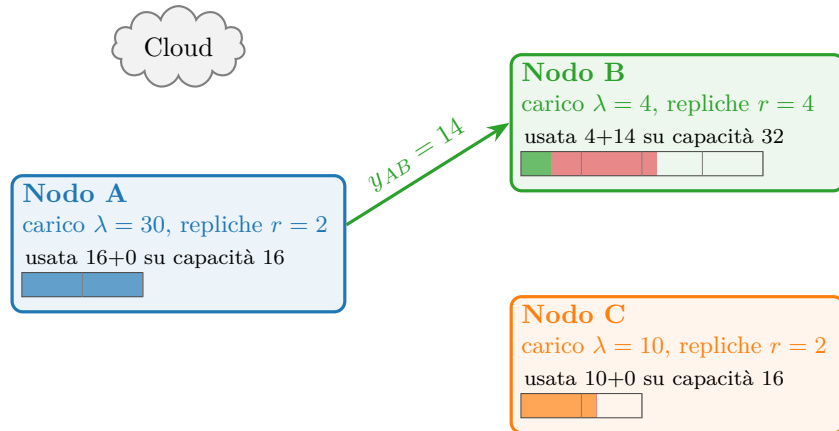
Mercato della memoria. B ha 3 unità libere e un'unica richiesta (da A): accende $\lfloor 3/1 \rfloor = 3$ nuove repliche, $r_B : 1 \rightarrow 4$. Ora B può sostenere 32 richieste/s. Φ non cambia (le repliche da sole non spostano richieste), ma il giro conta come “qualcosa è successo”, quindi si continua.



dopo il mercato: $r_B = 4$, $\Phi = 2.800$

Giro 2, mossa di A — riassegnazione verso il vicino migliore

- A ritira i propri inoltri; residui: $B = 32 - 4 = 28$, $C = 16 - 10 = 6$.
- $\bar{\omega}_A = 28 + 6 = 34$.
- Proposta: $x_A = 16$, $\omega_A = 14$ (tutto l'eccesso).
- B ha il β più alto e ora c'è posto: tutte e 14 vanno a B; C non riceve più nulla, il Cloud nemmeno.
- $u_A = \frac{16 + 0.9 \cdot 14}{30} = \frac{28.6}{30} = 0.953$; $\Delta u_A = 0.153 \Rightarrow$ **accettata**.
- Nessuna domanda scoperta: nessuna offerta.



$\Phi = 0.953 + 1 + 1 = 2.953$

Nota: A ha *spostato* 6 richieste da C a B. È consentito perché sono inoltri di A (li “possiede” lui); ciò che A non può fare è toccare quanto gli altri hanno promesso a lui.

Mosse di B e C. B continua a ricevere da A, quindi non può inoltrare: $\Delta u_B = 0$, rifiutata. C non riceve più nulla e potrebbe inoltrare fino a 14 richieste a B, ma eseguire in locale vale di più ($\alpha = 1 > \beta_{CB} = 0.9$): $\Delta u_C = 0$, rifiutata. Nessuno ha chiesto memoria: il mercato non accende repliche.

Giro 3 — certificato di equilibrio

A, B e C ripropongono la stessa scelta: tre $\Delta u = 0$, nessuna offerta, nessuna nuova replica. Condizione di stop soddisfatta: **equilibrio ε -Nash**, nessun nodo può migliorare da solo.

Riepilogo

Momento	y_{AB}	y_{AC}	z_A	r_B	u_A	Φ	esito
Avvio	0	0	14	1	0.487	2.487	—
Giro 1, mossa A	4	6	4	1	0.800	2.800	accettata, offerta a B
Giro 1, B e C	4	6	4	1	0.800	2.800	rifiutate
Giro 1, mercato	4	6	4	4	0.800	2.800	+3 repliche su B
Giro 2, mossa A	14	0	0	4	0.953	2.953	accettata
Giro 2, B e C	14	0	0	4	0.953	2.953	rifiutate
Giro 3	14	0	0	4	0.953	2.953	stop: equilibrio

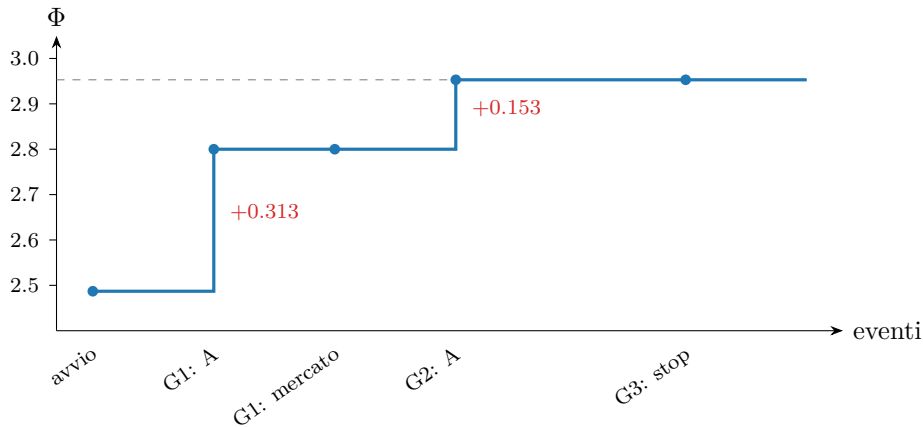


Figura 2: Il potenziale Φ sale a ogni mossa accettata e resta fermo altrimenti: per questo la ricerca termina.

5 Perché funziona (e perché si ferma)

- **Ogni mossa accettata alza Φ di più di ε .** Il nodo cambia solo cose sue (esecuzione locale, propri inoltri, proprie repliche) e non tocca le promesse altrui; quindi $\Delta\Phi = \Delta u_i$.
- **Φ è limitato** (al massimo ogni nodo esegue tutto con premio α), dunque le mosse accettate sono un numero finito.
- **Il mercato della memoria aggiunge solo repliche** dove c'è memoria libera: la memoria è finita, quindi anche questo si esaurisce.
- **Il divieto di ping-pong** toglie alcune mosse ma non rompe l'argomento: si sceglie sempre tra meno alternative.

- **Proposta e accettazione usano due misure diverse:** la proposta usa un prezzo medio δ per l'inoltro (veloce da calcolare), l'accettazione i veri β_{ij} . È l'accettazione a garantire che Φ salga, non l'ottimalità della proposta.

6 PG come raffinamento di MADeA (MADeA-PG)

Nel codice “PG” compare in due ruoli diversi.

PG da solo (FaaS-MAPG, `decentralized_potentialgame.py`, varianti S e R): è l'algoritmo descritto finora. Parte dalla soluzione egoista (dell'LSP tiene solo x e r ; l'inoltro calcolato dall'LSP viene scartato e si parte da $y = 0$) e arriva da solo all'equilibrio.

PG dopo l'asta (MADeA-PG, `madea_pg.py`; one-shot-PG, `one_shot_pg.py`): prima gira l'asta (MADeA iterata oppure one-shot) fino al suo criterio di stop; poi la sua *migliore soluzione* viene passata a `refine_solution`, che esegue giri di `node_move` *partendo da quella soluzione*. È quindi una ricerca locale a valle dell'asta.

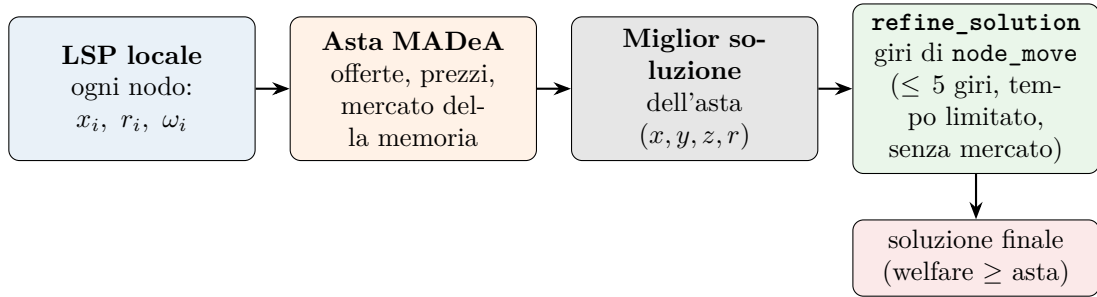


Figura 3: La catena di MADeA-PG (`run_faasmadea.py`, righe con `refine_welfare`).

Cosa cambia nella fase PG

	PG da solo	PG dopo l'asta (<code>refine_solution</code>)
Punto di partenza	soluzione egoista, $y = 0$	soluzione dell'asta, $y \neq 0$
Mercato della memoria	sì	no: si passa $\rho = 0$, quindi nessuna richiesta di memoria ai vicini; un nodo può solo ridecidere le <i>proprie</i> repliche
Limiti	numero di iterazioni, tempo	al più 5 giri (<code>max_sweeps</code>) e un budget di tempo (5 s, oppure 0.25 s per nodo in one-shot-PG)
Stop	equilibrio ε -Nash certificato	“nessuna proposta migliorante”: il codice dice esplicitamente che <i>non</i> è un certificato di Nash
Proposte	accettate se $\Delta u_i > \varepsilon$	in più validate (una proposta che viola i vincoli del nodo viene saltata)

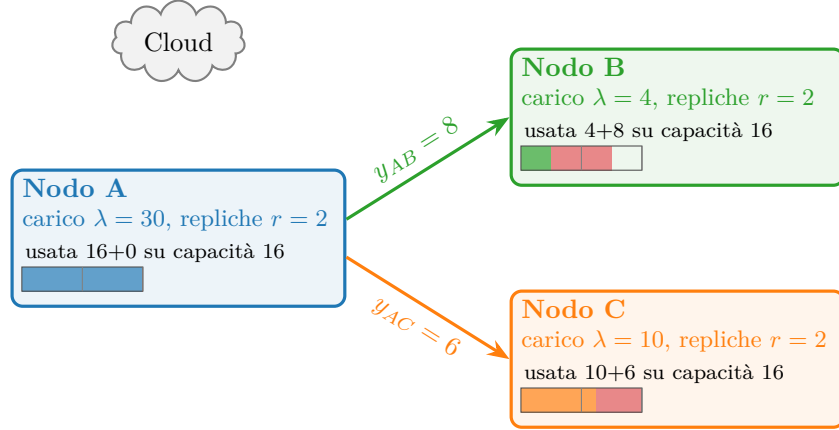
Ogni mossa accettata alza il welfare, quindi il risultato finale non può essere peggiore di quello dell'asta.

Esempio: la stessa rete, ripartendo dall'asta

Stessa istanza di prima (A, B, C). Le opzioni d'asta sono quelle degli esperimenti (`experiments/madea_pg_compact/config.json`): offerte unitarie, incremento $\epsilon_{\text{asta}} = 0.001$, prezzi che calano dell'1% quando nessuno compra ($\zeta = 0.01$). Anche qui i numeri vengono dall'esecuzione del codice (`run_madea_cycle` e poi `refine_solution`).

Fase 1: l'asta MADeA. L'LSP dà ad A $x_A = 16$ e una domanda da inoltrare $\omega_A = 14$.

It.	Cosa succede	y_{AB}	y_{AC}	r_B	W
0	A offre 4 unità a B (prezzo offerto 0.101) e 6 a C (0.001): tutte assegnate. Restano 4 unità da piazzare: A chiede memoria a B e il mercato accende 3 repliche ($r_B : 1 \rightarrow 4$).	4	6	4	2.800
1	Per A, B e C ora valgono uguale ($0.9 - 0.101 = 0.8 - 0.001$): offre le 4 unità a C, che però è pieno. Nulla assegnato. Il problema ristretto ricalcola le repliche sul carico effettivo e B torna a $r_B = 1$; il prezzo di B cala, quello di C sale.	4	6	1	2.800
2	Ora B è più conveniente: A gli offre le 4 unità; B accende al volo una replica e le accetta. Tutto il carico è assegnato: stop.	8	6	2	2.933



fine dell'asta: $W = 0.933 + 1 + 1 = 2.933$

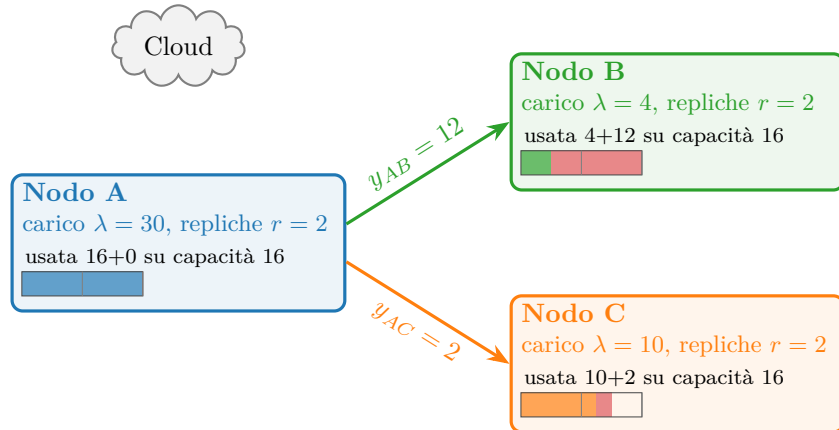
L'asta lascia 6 richieste su C ($\beta_{AC} = 0.8$) anche se B ($\beta_{AB} = 0.9$) avrebbe posto: A offre solo per la domanda *ancora da piazzare*, e i contratti già chiusi con C non vengono rimessi in discussione dal compratore.

Fase 2: refine_solution.

Giro 1, A: ritira i propri inoltri e legge i residui: B = $16 - 4 = 12$ (2 repliche), C = $16 - 10 = 6$; tetto $\bar{\omega}_A = 18$. Propone $\omega_A = 14$; la divisione golosa dà 12 a B ($\beta = 0.9$) e 2 a C.
 $u_A = \frac{16 + 0.9 \cdot 12 + 0.8 \cdot 2}{30} = \frac{28.4}{30} = 0.947$, $\Delta u_A = +0.013 \Rightarrow$ **accettata**.

Giro 1, B e C: B riceve da A (divieto di ping-pong), C preferisce eseguire in locale: rifiutate.

Giro 2: nessuna proposta migliora: stop ("no improving proposal").



dopo il raffinamento PG: $W = 0.947 + 1 + 1 = 2.947$

Confronto finale sull'esempio.

Metodo	y_{AB}	y_{AC}	r_B	Welfare W
MADeA (solo asta)	8	6	2	2.933
MADeA-PG (asta + raffinamento)	12	2	2	2.947
PG da solo	14	0	4	2.953

- **Cosa aggiunge PG all'asta:** il compratore può *spostare i propri inoltri* verso il vicino migliore (qui 4 richieste da C a B). L'asta non lo fa, perché offre solo per la domanda rimasta.
- **Perché qui MADeA-PG resta sotto PG da solo:** nel raffinamento il mercato della memoria è spento. B ha ancora 2 unità di memoria libere, ma nessuno può chiedergli nuove repliche; quindi A riesce a mettere su B solo 12 richieste invece di 14.
- Su questa istanza giocattolo vince PG da solo; su istanze reali il confronto può andare diversamente (gli esperimenti sono in `experiments/madea_pg_compact`).

7 Chi sa cosa: PG, one-shot e MADEA a confronto

Nel codice tutto gira in un unico processo e i nodi leggono matrici condivise; la tabella elenca quindi le informazioni che ogni nodo *usa*, cioè quelle che in un sistema distribuito dovrebbero viaggiare sulla rete. “One-shot” è l’asta MADEA a una passata (`decentralized_auction.py`); “MADEA” è l’asta iterata con riassegnazione (`run_faasmadea.py`). La colonna PG si riferisce a PG da solo: MADeA-PG scambia le informazioni della colonna MADEA e poi quelle della colonna PG, senza le richieste di memoria (mercato spento nel raffinamento).

Informazione	One-shot	MADEA	PG
Capacità residua del vicino, per funzione	✓ residuo vero (blackboard)	✓ capacità al netto del solo carico proprio del vicino; il residuo vero lo usa il venditore per riassegnare	✓ residuo vero (stessa funzione compute_residual_capacity)
Memoria libera ρ_j	✓	✓	✓
Prezzo p_{jf} annunciato dal venditore	✓	✓	×
Offerta del compratore (quantità + prezzo b)	✓	✓	×
Risposta del venditore (accetta/rifiuta, nuovo prezzo)	✓	✓ e può togliere carico a chi l'aveva prima	×
Richieste di memoria (i, j, f)	✓ solo se non ci sono offerte di capacità	✓ e il venditore può accendere repliche già durante l'asta	×
Divieto di ping-pong	il venditore guarda se il compratore <i>riceve</i> f	idem	×
Latenza, contatore di equità	dati propri del compratore	idem	×
Ritiro dei propri inoltri	×		
Turni	sincroni: tutti offrono insieme, poi tutti i venditori decidono	sincroni	×
Informazione globale per fermarsi	pesante: obiettivo centralizzato (miglior soluzione, “non migliora più”), “capacità finita”, “tutto assegnato”	idem	×

In sintesi.

- **Informazioni in più in PG:** nessuna. L’unica differenza è che il sì/no “sta inoltrando f ?” lo legge il compratore invece del venditore.
- **Informazioni in meno in PG:** prezzi, valori delle offerte, risposte dei venditori, latenza ed equità, obiettivo globale per fermarsi.
- **Il costo di PG è il coordinamento, non i dati.** L’asta è parallela, con due messaggi per iterazione; PG richiede un turno condiviso e stato fresco dei vicini, cioè N mosse in sequenza per giro. Inoltre presuppone che il vicino accetti carico entro il residuo che ha annunciato, senza poter rifiutare.

- **One-shot-PG** (`one_shot_pg.py`) è l'asta one-shot seguita da un affinamento PG (`madea_pg.refine_solution`, che usa `node_move`): scambia l'unione delle due colonne, in due fasi.

8 Dove si trova nel codice

Passo	Funzione (<code>decentralized_potentialgame.py</code>)
ciclo esterno, avvio, stop	<code>_run, check_pg_stopping</code>
memoria libera ρ	<code>compute_rho</code>
giro dei nodi (S / R)	<code>potential_game_sweep (run_pg_s, run_pg_r)</code>
mossa di un nodo (2a-2g)	<code>node_move</code>
proposta locale (2d)	<code>propose_node_move</code> $\rightarrow$ LSP_pg / DP
divisione golosa (2e)	<code>split_omega</code>
guadagno e Cloud	<code>compute_node_utility, compute_z</code>
mercato della memoria (3)	<code>start_additional_replicas (run_faasmadea.py)</code>
MADeA-PG, one-shot-PG	<code>refine_solution (madea_pg.py), one_shot_pg.py</code>